```
...

const EXPECTED_BODY = { id: 1, name: "My movie", year: 1999 };

const testId = 100;
EXPECTED_BODY.id = testId;

...

provider
  .given('a movie with specific ID', {     #1
    id: 100,                               #1
  })                                       #1
  .uponReceiving('a request to a specific movie')
  .withRequest({
    method: 'GET',
    path: '/movie/100',
  })
  .willRespondWith({
    status: 200,
    body: {
      id: integer(testId),
      name: string(EXPECTED_BODY.name),
      year: integer(EXPECTED_BODY.year)
    }
  })
})
...

stateHandlers: {                                    #2
    'a movie with specific ID': (parameters) => {   #2
      movies.getFirstMovie().id = parameters.id;    #2
      return Promise.resolve({ description: `Movie with ID
${parameters.id} added!` });         #2
    }                                               #2
  }                                                 #2

...
```

#1 Sets up consumer contract test with a provider state
#2 Sets up the provider state on the provider side

We explain this code in more detail in section 6.3.2, but briefly, the state `'a movie with specific ID'` ensures that the provider is in the right setup before verifying the contract. In the preceding example code, we're making sure that the movie ID the consumer has specified as part of the contract exists on the provider side. When the provider replays the interaction from the contract, the provider will have the movie ID the consumer wants to test.